

Task 1 — Get the Data Into a Database

MLOps Training 2026/2027

Submitted by: Fagr Ahmed Abdelrazek

Tool used: Kaggle Notebook + SQLite

Kaggle Data Set

https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce?resource=download&select=olist_geolocation_dataset.csv

Kaggle Notebook

<https://www.kaggle.com/code/fagrahmed/ml-ops-qafza-task-1-brazilian-e-commerce>

في هذا التقرير، تم تحميل بيانات Olist Brazilian E-Commerce Dataset، فهم هيكل الجداول والعلاقات بينها، بناء قاعدة بيانات علائقية باستخدام SQLite، والتأكد من إمكانية تنفيذ استعلامات وربط (joins) بين الجداول بنجاح

Step 1 — Read & Understand

- تم قراءة الـ PDF بنوع الـ Data Set وفهمت الملفات

Key	Links
order_id	orders, order_items, order_payments, order_reviews
customer_id	orders, customers
product_id	order_items, products
seller_id	order_items, sellers
zip_code_prefix	customers/sellers, geolocation

المشكلة المطلوب حلها هي "Late Delivery Classification" — أي التنبؤ بما إذا كان الطلب سيصل بعد الموعد المتوقع للتسليم (**order_estimated_delivery_date**) أم لا. تم توضيح خطر تسرب البيانات (Data Leakage) حيث لا يجوز استخدام **order_delivered_customer_date** كمُدخل (feature) للنموذج لاحقاً، لأنها معلومة غير متاحة وقت التنبؤ الفعلي.

Step 2 — Download & Ingest

- تم تحميل ملفات CSV مباشرة داخل بيئة Kaggle Notebook (مربوطة تلقائيًا بال Data Set)
- تم إنشاء قاعدة بيانات SQLite باسم `olist.db`
- تم تحميل كل ملف CSV كجدول منفصل داخل القاعدة

```
conn = sqlite3.connect("/kaggle/working/olist.db")

files_map = {
    "orders": "olist_orders_dataset.csv",
    "customers": "olist_customers_dataset.csv",
    "order_items": "olist_order_items_dataset.csv",
    "order_payments": "olist_order_payments_dataset.csv",
    "order_reviews": "olist_order_reviews_dataset.csv",
    "products": "olist_products_dataset.csv",
    "sellers": "olist_sellers_dataset.csv",
    "geolocation": "olist_geolocation_dataset.csv",
    "category_translation": "product_category_name_translation.csv",
}

for table_name, filename in files_map.items():
    path = f"{base_path}/{filename}"
    df = pd.read_csv(path)
    df.to_sql(table_name, conn, if_exists="replace", index=False)
    print(f"✓ {table_name}: {df.shape[0]} rows, {df.shape[1]} columns")

✓ orders: 99441 rows, 8 columns
✓ customers: 99441 rows, 5 columns
✓ order_items: 112650 rows, 7 columns
✓ order_payments: 103886 rows, 5 columns
✓ order_reviews: 99224 rows, 7 columns
✓ products: 32951 rows, 9 columns
✓ sellers: 3095 rows, 4 columns
✓ geolocation: 1000163 rows, 5 columns
✓ category_translation: 71 rows, 2 columns
```

تم تحميل 9 جداول بنجاح، كما هو موضح في الصورة، بإجمالي عدد صفوف يطابق الأعداد المذكورة في وصف الداتا سيت الأصلي على Kaggle.

Step 3 — Test It

- تم الاتصال بقاعدة البيانات وتنفيذ عدة استعلامات SQL للتأكد من سلامة البيانات
- تم اختبار الجداول بشكل فردي (SELECT)
- تم اختبار الربط بين جدولين وأكثر (JOIN)
- تم توضيح مشكلة تكرار الصفوف (Row Duplication) عند الربط مع `order_items` بدون تجميع (aggregation)، وحلها باستخدام `GROUP BY`

```

query3 = """
SELECT
    o.order_id,
    o.order_status,
    o.order_estimated_delivery_date,
    o.order_delivered_customer_date,
    COUNT(DISTINCT oi.product_id) as n_products,
    SUM(oi.price) as total_price,
    SUM(op.payment_value) as total_payment
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN order_payments op ON o.order_id = op.order_id
GROUP BY o.order_id
LIMIT 10
"""

pd.read_sql(query3, conn)

```

	order_id	order_status	order_estimated_delivery_date	order_delivered_customer_date	n_products	total_price	total_payment
0	00010242fe8c5a6d1ba2dd792cb16214	delivered	2017-09-29 00:00:00	2017-09-20 23:43:48	1	58.90	72.19
1	00018f77f2f0320c557190d7a144bdd3	delivered	2017-05-15 00:00:00	2017-05-12 16:04:24	1	239.90	259.83
2	000229ec398224ef6ca0657da4fc703e	delivered	2018-02-05 00:00:00	2018-01-22 13:19:16	1	199.00	216.87
3	00024acbcd0a6daa1e931b038114c75	delivered	2018-08-20 00:00:00	2018-08-14 13:32:39	1	12.99	25.78
4	00042b26cf59d7ce69dfabb4e55b4fd9	delivered	2017-03-17 00:00:00	2017-03-01 16:42:31	1	199.90	218.04
5	00048cc3ae777c65dbb7d2a0634bc1ea	delivered	2017-06-06 00:00:00	2017-05-22 13:44:35	1	21.90	34.59
6	00054e8431b9d7675808bcb819fb4a32	delivered	2018-01-04 00:00:00	2017-12-18 22:03:38	1	19.90	31.75
7	000576fe39319847cbb9d288c5617fa6	delivered	2018-07-25 00:00:00	2018-07-09 14:04:07	1	810.00	880.75
8	0005a1a1728c9d785b8e2b08b904576c	delivered	2018-03-29 00:00:00	2018-03-29 18:17:31	1	145.95	157.60
9	0005f0442cb953dcd1d21e1fb923495	delivered	2018-07-23 00:00:00	2018-07-04 17:28:31	1	53.99	65.39

```

query2 = """
SELECT
    o.order_id,
    o.order_estimated_delivery_date,
    o.order_delivered_customer_date,
    oi.product_id,
    oi.seller_id,
    oi.price,
    op.payment_type,
    op.payment_installments
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN order_payments op ON o.order_id = op.order_id
LIMIT 10
"""

pd.read_sql(query2, conn)

```

	order_id	order_estimated_delivery_date	order_delivered_customer_date	product_id	seller_id	price	payment
0	e481f51cbdc54678b7cc49136f2d6af7	2017-10-18 00:00:00	2017-10-10 21:25:13	87285b34884572647811a353c7ac498a	3504c0cb71d7fa48d967e0e4c94d59d9	29.99	credit
1	e481f51cbdc54678b7cc49136f2d6af7	2017-10-18 00:00:00	2017-10-10 21:25:13	87285b34884572647811a353c7ac498a	3504c0cb71d7fa48d967e0e4c94d59d9	29.99	credit
2	e481f51cbdc54678b7cc49136f2d6af7	2017-10-18 00:00:00	2017-10-10 21:25:13	87285b34884572647811a353c7ac498a	3504c0cb71d7fa48d967e0e4c94d59d9	29.99	credit
3	53cdb2fc8bc7dce0b6741e2150273451	2018-08-13 00:00:00	2018-08-07 15:27:45	595fac2a385ac33a80bd5114aec74eb8	289cdb325fb7e7f891c38608bf9e0962	118.70	credit
4	47770eb9100c2d0c44946d9cf07ec65d	2018-09-04 00:00:00	2018-08-17 18:06:29	aa4383b373c6aca5d8797843e5594415	4869f7a5dfa277a7dca6462dcf3b52b2	159.90	credit
5	949d5b44dbf5de918fe9c16f97b45f8a	2017-12-15 00:00:00	2017-12-02 00:28:42	d0b61bfb1de832b15ba9d266ca96e5b0	66922902710d126a0e7d26b0e3805106	45.00	credit
6	ad21c59c0840e6cb83a9ceb5573f8159	2018-02-26 00:00:00	2018-02-16 18:17:02	65266b2da20d04dbe00c5c2d3bb7859e	2c9e548be18521d1c43cde1c582c6de8	19.90	credit
7	a4591c265e18cb1dcee52889e2d8acc3	2017-08-01 00:00:00	2017-07-26 10:57:55	060cb19345d90064d1015407193c233d	8581055ce74af1daba164fdbd55a40de	147.90	credit
8	136cce7faa42fdb2cefd53fdc79a6098	2017-05-09 00:00:00	None	a1804276d9941ac0733cdf409f5206eb	dc8798cbf453b7e0f98745e396cc5616	49.90	credit
9	6514b8ad8028c9f2cc2374ded245783f	2017-06-07 00:00:00	2017-05-26 12:55:51	4520766ec412348b8d4caa5e8a18c464	16090f2ca825584b5a147ab24aa30c86	59.99	credit

```
query1 = """
SELECT
    o.order_id,
    o.order_status,
    o.order_purchase_timestamp,
    o.order_estimated_delivery_date,
    o.order_delivered_customer_date,
    c.customer_city,
    c.customer_state
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
LIMIT 10
"""
pd.read_sql(query1, conn)
```

	order_id	order_status	order_purchase_timestamp	order_estimated_delivery_date	order_delivered_customer_date	customer_city	customer_state
0	e481f51cbdc54678b7cc49136f2d6af7	delivered	2017-10-02 10:56:33	2017-10-18 00:00:00	2017-10-10 21:25:13	sao paulo	SP
1	53cdb2fc8bc7dce0b6741e2150273451	delivered	2018-07-24 20:41:37	2018-08-13 00:00:00	2018-08-07 15:27:45	barreiras	BA
2	47770eb9100c2d0c44946d9cf07ec65d	delivered	2018-08-08 08:38:49	2018-09-04 00:00:00	2018-08-17 18:06:29	vianopolis	GO
3	949d5b44dbf5de918fe9c16f97b45f8a	delivered	2017-11-18 19:28:06	2017-12-15 00:00:00	2017-12-02 00:28:42	sao goncalo do amarante	RN
4	ad21c59c0840e6cb83a9ceb5573f8159	delivered	2018-02-13 21:18:39	2018-02-26 00:00:00	2018-02-16 18:17:02	santo andre	SP
5	a4591c265e18cb1dcee52889e2d8acc3	delivered	2017-07-09 21:57:05	2017-08-01 00:00:00	2017-07-26 10:57:55	congoninhas	PR
6	136cce7faa42fdb2cef53fdc79a6098	invoiced	2017-04-11 12:22:08	2017-05-09 00:00:00	None	santa rosa	RS
7	6514b8ad8028c9f2cc2374ded245783f	delivered	2017-05-16 13:10:30	2017-06-07 00:00:00	2017-05-26 12:55:51	nilopolis	RJ
8	76c6e866289321a7c93b82b54852dc33	delivered	2017-01-23 18:29:09	2017-03-06 00:00:00	2017-02-02 14:08:10	faxinalzinho	RS
9	e69bfb5eb88e0ed6a785585b27e16dbf	delivered	2017-07-29 11:55:02	2017-08-23 00:00:00	2017-08-16 17:14:30	sorocaba	SP

- ✓ Database is running locally with the data inside
- ✓ Tables can be queried and joined successfully
- ✓ Table relationships are understood (order_id, customer_id, product_id, seller_id)
- ✓ The problem to solve is understood (Late Delivery Classification)